LAB 2_1B

# Power Analysis for Password Bypass

Student Guide - ChipWhisperer-Husky hardware edition

How to run the lab, check your own password guesses on the real chip, and change the password the chip is protecting - written so that nobody needs any prior experience.

> **HOW TO USE THIS GUIDE**
>
> This guide goes **with** the Jupyter notebooks - it does not replace them. Keep the notebook open next to this PDF and follow along. Every step below says what to do, what you should see, and what to try if you do not see it.

## Which files are which

| File (folder: jupyter/courses/sca101/) | What it is | Open it? |
|---|---|---|
| `Lab 2_1B - Power Analysis for Password Bypass (MAIN).ipynb` | The lab itself. Everything you do happens here. | **Yes** |
| `SOLN_Lab 2_1B - Power Analysis for Password Bypass.ipynb` | Answer key. Peek only if you are really stuck. | If stuck |
| `Lab 2_1B - Firmware Change and Flash Walkthrough.ipynb` | Extra, detailed walkthrough of changing the password (Part 4 of this guide). | Optional |
| `Lab 2_1B ... (HARDWARE).ipynb` and `(SIMULATED).ipynb` | Back-end helpers that the MAIN notebook runs for you. | **No** |
| `Lab 0 - SCA101 Setup.ipynb` | One-time hardware and software check. | Once, first |

## The four colored boxes you will see everywhere

**DO THIS**
An action to take.

**YOU SHOULD SEE**
What the screen should show.

**IF IT DOESN'T**
What to try if it didn't.

**WATCH OUT**
A mistake that costs time.

Guide version 1.0 - October 2026. Written for ChipWhisperer-Husky + CW313 + CW312T-SAM4S (ATSAM4S2AA) hardware.

# Contents

**The big picture . . . 3**

**Before you start . . . 4**

**Part 1 - One-time setup . . . 4**

1.1 Assemble the hardware . . . 4

1.2 Start Jupyter and open Lab 0 . . . 5

1.3 Run the Lab 0 checks . . . 5

**Part 2 - Walk through the lab . . . 6**

2.1 Set the options . . . 6

2.2 Connect, build, and flash . . . 6

2.3 Run the sanity check . . . 7

2.4 Look at one correct and one wrong character . . . 7

2.5 Plot every possible character . . . 7

2.6 Subtract a reference to make the outlier obvious . . . 8

2.7 Give each character a score . . . 9

2.8 Recover the whole password . . . 9

**Part 3 - Check your own guesses . . . 11**

3.1 Run the helper cell . . . 11

3.2 Use the guess box . . . 11

3.3 Experiments to try . . . 12

3.4 Lock it in . . . 12

**Part 4 - Change the password . . . 13**

4.1 Path A - the easy way (recommended) . . . 13

4.2 Path B - rebuild and reflash by hand (no notebook) . . . 14

4.3 Putting the original password back . . . 15

4.4 Using a different length (advanced) . . . 15

**Troubleshooting . . . 16**

**You are done when... . . . 17**

**Appendix A - Quick reference . . . 18**

**Appendix B - Glossary . . . 18**

**Appendix C - Where things live . . . 19**

# The big picture

**What is this lab about?** A simple password checker compares your guess with the real password one character at a time and **stops at the first wrong character**. That means a guess with more correct leading characters makes the chip do slightly *more work* before it gives up - and more work means a slightly different **power draw**.

The ChipWhisperer measures the chip's power while it checks a guess; the recording is called a **power trace**. By comparing traces you can see which character was right, lock it in, and move on to the next position. After five rounds you have the whole password - **without ever being told it**. This is called *simple power analysis* (SPA).

### How the chip checks a guess (the early exit is the leak):

| Real password | h | 0 | p | x | 3 | Result |
|---|---|---|---|---|---|---|
| **Guess** `zzzzz` | z: X wrong | - | - | - | - | Stops after 1 comparison |
| **Guess** `h0zzz` | h: ok | 0: ok | z: X wrong | - | - | Stops after 3 comparisons |
| **Guess** `h0px3` | ok | ok | ok | ok | ok | All match: access granted |

(The table uses the lab's stock password `h0px3` as the example.)

### Your path through the guide:

**1 Set up** (one time) → **2 Run the lab** (find the password) → **3 Check guesses** (ask the real chip) → **4 Change password** (optional)

| Part | What you do | Where |
|---|---|---|
| **1 Set up** | Build the hardware, start Jupyter, run the Lab 0 checks. | Lab 0 notebook |
| **2 Run the lab** | Capture power traces and recover the password automatically. | MAIN notebook, top to bottom |
| **3 Check guesses** | Type any guess into a box and see if the real chip accepts it. | MAIN notebook, 'Hands-On' section |
| **4 Change the password** | Edit the chip's firmware, rebuild, reflash, attack the new password. | A text editor + the notebook |

# Before you start

Tick every row before Part 1. Most problems later come from something on this list.

| | You need | How to check |
|---|---|---|
| [ ] | **ChipWhisperer-Husky** capture board with its USB-C cable (use a data cable, not a charge-only one). | It lights up when plugged in. |
| [ ] | **CW313** base board with the **CW312T-SAM4S** target (the ATSAM4S2AA microcontroller board). | The label on the small board says SAM4S. It is *not* the iCE40 FPGA board that may be in the same box. |
| [ ] | **20-pin ribbon cable** and **two SMA cables** from the kit. | Used in Part 1, Step 1. |
| [ ] | A computer with the **ChipWhisperer Python environment**, **Jupyter**, and the **ARM compiler** (`arm-none-eabi-gcc`) plus `make`. | Lab 0 tests all of this for you (Part 1, Step 3). |
| [ ] | The course repository, with the `jupyter/` and `firmware/` folders **next to each other**. | The notebooks reach the firmware through `../../../firmware/`. If you moved only `jupyter/`, the lab cannot build. |
| [ ] | A text editor (Notepad, VS Code, ...). Only needed for Part 4. | Any editor works. |

> **WATCH OUT**
>
> **Only one program can talk to the Husky at a time.** If a second notebook, an old script, or a forgotten Jupyter tab still has it open, the next connection fails with `Unable to communicate with found ChipWhisperer`. Keep exactly **one** notebook connected, and shut the others down (Kernel > Shut Down) before starting another.

# Part 1 - One-time setup

## 1.1 Assemble the hardware

1. Work on a static-free surface. The boards are bare.
2. Press the **CW312T-SAM4S** target onto the **CW313** through its two board-to-board connectors. It only fits one way; push until flush.
3. Connect the CW313's **20-pin header** to the Husky's **20-pin connector** with the ribbon cable.
4. Connect two SMA cables, finger-tight only: Husky **Measure** to CW313 **MEASURE** (this carries the power trace), and Husky **Glitch/ClkIn** to CW313 **GLITCH/HS2** (the target's clock).
5. Check the CW313 jumpers: **JP2** on **T-GPIO4** (carries the trigger) and **JP1** and **JP4** connected. These are the factory settings on a new board.
6. Plug the USB-C cable from the Husky into your computer. The Husky's power light comes on. The target is powered over the ribbon - no separate supply.

> **WATCH OUT**
>
> The **Measure** SMA cable matters more than it looks. If it is loose or missing, the lab still runs, but the power traces are tiny and the attack recovers *random-looking* characters. If your result looks like gibberish, check this cable first (see Troubleshooting T-10).

The full assembly photo and details are in the root notebook `1 - Connecting to Hardware.ipynb` (section 'CW-Husky with CW313/SAM4S').

## 1.2 Start Jupyter and open Lab 0

**DO THIS**

Open a terminal in the **ChipWhisperer environment** (on Windows this is usually the 'CW Compiler Environment' shell, or whatever shell your instructor set up), go to the `jupyter` folder, and start Jupyter:

```
cd chipwhisperer/jupyter
jupyter lab
```

A browser tab opens (normally at `http://localhost:8888`). If your instructor gave you a ready-made Jupyter link instead, just use that. In the file list on the left, open `courses` > `sca101` > `Lab 0 - SCA101 Setup.ipynb`.

**GOOD TO KNOW**

**Jupyter in 30 seconds.** A notebook is a page made of *cells*. Click a cell, then press **Shift+Enter** to run it and jump to the next one. While a cell runs, its left margin shows `[*]`; when finished it shows a number like `[3]`. Always run cells **top to bottom**, and wait for `[*]` to disappear before running the next. The *kernel* is the Python process behind the notebook; restarting it (Kernel menu) wipes everything and lets you start clean.

## 1.3 Run the Lab 0 checks

In Lab 0, run each code cell in order. Each must finish with no red error text.

| Check | What it proves |
|---|---|
| 1. `import chipwhisperer` | The ChipWhisperer software is installed. It prints a version number. |
| 2. Plotting stack | The libraries used for every trace plot are present. |
| 3. `make` and `arm-none-eabi-gcc` | The compiler that turns the lab's C code into chip firmware is installed. Needed for the hardware labs. |
| Hardware connection test | Run only with the Husky plugged in. It connects, prints details, and disconnects. Expect a line starting `connected:` and then an 'hardware OK' line. |

**IF IT DOESN'T**

**Unable to open USB device** (Windows): install the Husky driver with the **Zadig** tool (WinUSB driver for 'ChipWhisperer Husky'), then retry. Also try a different USB-C cable or port.

**Check 3 fails**: the compiler tools are not on your PATH - start Jupyter from the ChipWhisperer compiler environment, or ask your instructor.

More fixes are in the Troubleshooting table near the end.

**WATCH OUT**

Before moving on, **shut down Lab 0's kernel** (Kernel > Shut Down Kernel) or close its tab so it lets go of the Husky. The hardware test already disconnects, but a half-finished run might not.

# Part 2 - Walk through the lab

Open `Lab 2_1B - Power Analysis for Password Bypass (MAIN).ipynb`. Read the text in each cell as you go - it is short, and it explains the idea behind the code. Run the code cells in order with Shift+Enter. This part follows the notebook's own sections.

## 2.1 Set the options

Find the cell under **'Choose your setup options here'**. For the Husky kit it must read:

```
SCOPETYPE = 'OPENADC'      # 'OPENADC' for CW-Lite/CW-Pro/CW-Husky, 'CWNANO' for CW-Nano
PLATFORM  = 'CWHUSKY'      # SAM4S target in the CW-Husky kit
CRYPTO_TARGET = 'NONE'
SS_VER = 'SS_VER_2_1'

VERSION = 'HARDWARE'      # 'SIMULATED' (pre-recorded traces) or 'HARDWARE' (live capture)
```

| Setting | In plain English |
|---|---|
| `SCOPETYPE`, `PLATFORM` | Which capture board and which target chip you have. Leave as shown for the Husky kit. |
| `CRYPTO_TARGET`, `SS_VER` | Compile-time options for the firmware. Leave as shown. |
| `VERSION` | **The one to check.** `'HARDWARE'` = use your real board. `'SIMULATED'` = replay recorded traces (no board needed, but the Hands-On section and password changes only work with real hardware). |

**DO THIS**

Make sure `VERSION = 'HARDWARE'`, then run the cell. It prints nothing.

## 2.2 Connect, build, and flash

The next cell starts with `%run`. Run it. It does four things for you:

- connects to the Husky;
- **compiles** the password-checking C program (`basic-passwdcheck.c`) into chip code;
- **flashes** (writes) that code onto the SAM4S chip;
- defines `cap_pass_trace(guess)`: reset the chip, send your guess plus Enter, and return the power trace recorded while it checks.

**YOU SHOULD SEE**

A long scroll of text. Look for `INFO: Found ChipWhisperer`, then compiler lines ending in `Built for platform CW-Husky Microchip SAM4S`, then programming messages. It is finished when `[*]` becomes a number. Be patient - compiling takes a moment.

**GOOD TO KNOW**

This cell **rebuilds and reflashes every time it runs**. That is exactly why changing the password later (Part 4) is easy: edit one line of C, then re-run this cell.

**IF IT DOESN'T**

See Troubleshooting T-1 to T-9 (connection, build, flash).

## 2.3 Run the sanity check

Run the small cell that calls `cap_pass_trace("h\n")` and checks the trace length.

**YOU SHOULD SEE**

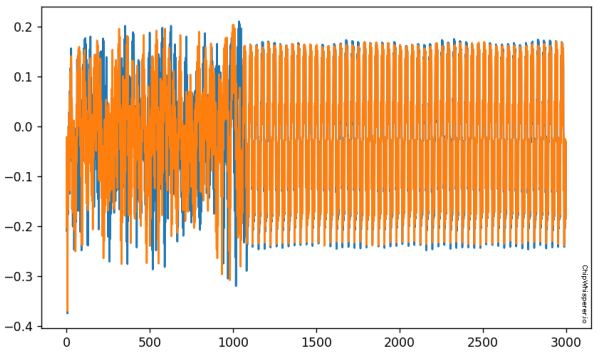
A line reading **OK to continue!** (the notebook puts a check mark in front). Do not go on until you see it. You may also see a warning `SAM3U Serial buffers OVERRUN`; if the OK line still follows, carry on. If the warning keeps appearing or things fail, use a different USB cable or port.

## 2.4 Look at one correct and one wrong character

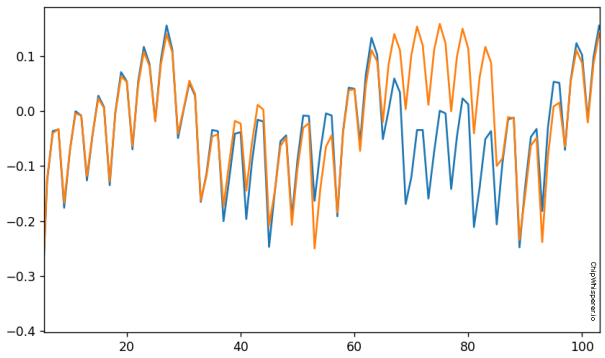
The next code cell captures a trace for the guess `h` and one for `0`, and draws both on one plot:

```
trace_h = cap_pass_trace("h\n")
cw.plot(cap_pass_trace("h\n")) * cw.plot(cap_pass_trace("0\n"))
```

The `\n` at the end of a guess is the Enter key - the chip treats it as 'I'm done typing'. Plots are interactive: use the toolbar to box-zoom, scroll-zoom, and reset.




Both traces, whole recording.



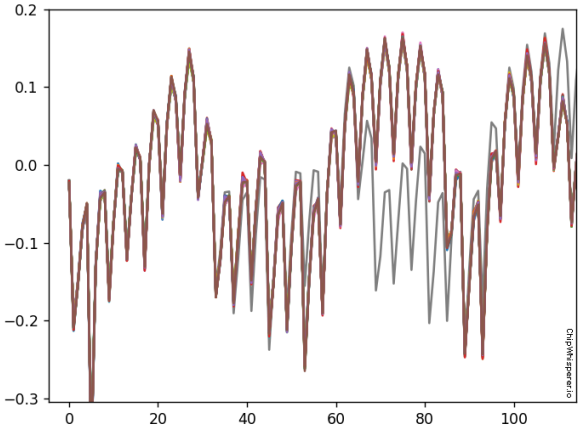

Zoomed to the start: the two traces split apart.

**YOU SHOULD SEE**

Two traces that look almost identical, but **separate** partway through. That split is the chip taking a different path for a correct character than for a wrong one. Your picture will be close to, but not exactly, the reference image (every board is a bit different).

## 2.5 Plot every possible character

The next cell loops over all 36 allowed characters (`a-z`, `0-9`), captures a trace for each, and plots the first 1000 samples of all of them. A progress bar counts through the 36 captures.




35 traces bunch together; one (grey here) leaves the pack. That one is the right first character.

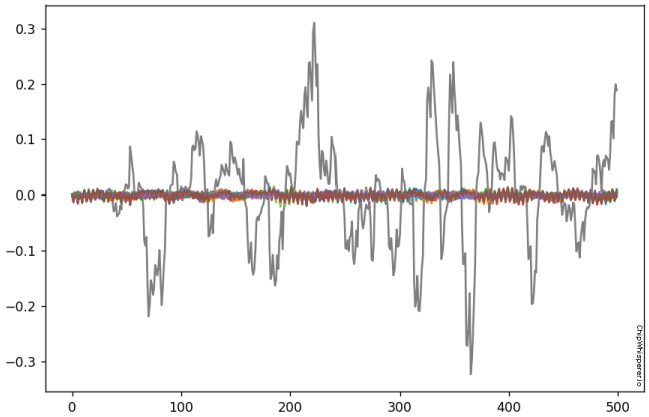
**YOU SHOULD SEE**

One **outlier** trace that does not follow the others. The traces are not labelled, which is fine - the next steps find the outlier automatically.

## 2.6 Subtract a reference to make the outlier obvious

To make the difference stand out, the notebook records a **reference** trace using `\x01` (an invalid character - it is processed like any wrong guess), then plots every character's trace *minus* the reference.

```
ref_trace = cap_pass_trace("\x01\n")[0:1000]
...
fig *= cw.plot(trace - ref_trace)
```




Wrong characters collapse to a flat line near zero; the right character is the big wiggly one.

**YOU SHOULD SEE**

35 lines hugging zero and **one** big swinging line. (A line near zero = 'same as the wrong-guess reference'.)

## 2.7 Give each character a score

Looking at plots does not scale, so the notebook turns each difference into **one number**: `np.sum(np.abs(trace - ref_trace))` - subtract, make every value positive, add them all up. A small number = 'looks like a wrong guess'; a large number = 'behaves differently'. Run the three tiny `np.abs` / `np.sum` demo cells first (they just show what those functions do), then the scoring cell.

> **YOU SHOULD SEE**
>
> A list like `a diff = ...` down to `9 diff = ...`. **One** character has a much larger number than the rest. In one of our test runs on a Husky, the correct character scored about 336 while wrong ones scored roughly 10 to 62. Your exact numbers will differ - what matters is that one stands far apart.

## 2.8 Recover the whole password

Now do the same trick for each position, building the password one character at a time. Run the cell under **'Running a Full Attack'**:

```
guessed_pw = ""          # known-good prefix so far

for _ in range(5):        # password is 5 characters long
    ref_trace = cap_pass_trace(guessed_pw + "\x01\n")
    best_diff, best_char = 0, '?'

    for c in 'abcdefghijklmnopqrstuvwxyz0123456789':
        trace = cap_pass_trace(guessed_pw + c + "\n")
        diff = np.sum(np.abs(trace - ref_trace))
        if diff > best_diff:
            best_diff, best_char = diff, c

    guessed_pw += best_char
    print(guessed_pw)
```

### In words, each of the 5 rounds does this:

1. Record a **reference**: the characters found so far, followed by a deliberately wrong character.
2. Try all 36 characters in the next position, each after the characters found so far.
3. Score each against the reference; the **highest score** is the character the chip liked.
4. Add that character to `guessed_pw`, print it, and repeat for the next position.

### For the stock password the rounds go like this:

| Round | Known so far | Trying | Winner |
|---|---|---|---|
| 1 | (nothing) | a..9 in position 1 | h |
| 2 | h | ha, hb, ... h9 | 0 |
| 3 | h0 | h0a ... h09 | p |
| 4 | h0p | h0pa ... h0p9 | x |
| 5 | h0px | h0pxa ... h0px9 | 3 |

**YOU SHOULD SEE**

The password grows one character per line. With the stock password:

```
h
h0
h0p
h0px
h0px3
```

The cell makes about 185 captures (37 per round x 5 rounds), so it takes a while. Do **not** interrupt it - wait until `[*]` becomes a number.

**GOOD TO KNOW**

If your instructor (or you, in Part 4) changed the password, you will see *that* password build up instead of `h0px3`. The method is identical.

### Finally, run the check cell. It confirms you recovered the password and prints it:

```
assert guessed_pw == 'h0px3', "Password recovery failed - see the troubleshooting ..."
print("recovered password:", guessed_pw)
```

**YOU SHOULD SEE**

No error, and a `recovered password:` line. **You have broken the password.**

**IF IT DOESN'T**

**AssertionError: Password recovery failed.** Either the attack picked a wrong character (see T-10 to T-15), or the chip's password is not `h0px3` because someone changed it. If the printed password looks like a perfectly sensible password, it was changed - update the `assert` (Part 4, step 5).

# Part 3 - Check your own guesses

The attack *infers* the password from power. It never actually **asks the chip** whether the answer is right. This section closes the loop: you type a guess, the real chip tells you if it unlocks, and you see the power trace of that guess.

> **WATCH OUT**
> **Hardware only.** This section talks to the chip over its serial connection. With `VERSION = 'SIMULATED'` it just prints 'skipped - hardware only' (that is intentional, not a bug).

What the chip says. After checking a guess the firmware prints one of two messages and then freezes until it is reset (the notebook resets it for you on every guess):

| Chip prints | Meaning | Notebook shows |
|---|---|---|
| `Access granted, Welcome!` | Your guess is the password. | `ACCESS GRANTED`, device said: `Access \| granted, \| Welcome!`, **green** trace |
| `PASSWORD FAIL` | Not the password. | `password fail`, device said: `PASSWORD \| FAIL`, **red** trace |

(The notebook puts a `|` between the words of the chip's reply just to make it easy to read.)

## 3.1 Run the helper cell

Under **'Hands-On: Check Your Own Guess'**, read the 'Before you start' box, then run the cell that defines `try_password()`. It also does one quick text-only self-test.

> **YOU SHOULD SEE**
> Either a `guess = '...' -> ...` line, or - if `VERSION` is not `'HARDWARE'` - the message 'Hands-on check skipped'.

> **IF IT DOESN'T**
> `RuntimeError: try_password() needs the ChipWhisperer connected` means `VERSION` was not `'HARDWARE'`, or the setup cell (2.2) was never run. Fix that and re-run from the top.

## 3.2 Use the guess box

Run the next cell. It prints a verdict for the starting guess, then shows an interactive box:

| Element | What to do |
|---|---|
| Text box labelled **Guess:** | Pre-filled with the password the attack found. Click in it and type any guess (letters and digits). |
| Blue **Try it** button (padlock icon) | Click to send your guess to the chip. Wait about a second. |
| Area below the button | Shows the verdict line, what the device said, and the power trace for that guess. |

**GOOD TO KNOW**

Every guess is independent - the chip is reset each time - so you can click as often as you like. The plot shows the first 1000 samples of the trace (green = accepted, red = rejected).

**IF IT DOESN'T**

**Box and button don't appear / blank area:** restart the kernel (Kernel > Restart), refresh the browser tab, run the notebook again from the top. On very old classic Jupyter you may need `jupyter nbextension enable --py widgetsnbextension`.

`ModuleNotFoundError: No module named 'ipywidgets'`**:** run `pip install ipywidgets` in the ChipWhisperer environment, restart the kernel.

## 3.3 Experiments to try

These make the idea stick. Predict the result first, then check:

1. Type the recovered password. Expect **ACCESS GRANTED** and a green trace.
2. Change just the **last** character. Expect `password fail`.
3. Compare the traces for a guess with the first **4** characters right and the 5th wrong against the fully correct guess. Where do they start to differ? (Hint: think about how many characters the chip compared before it stopped.)
4. Try a completely wrong guess and a guess with a wrong *first* character. Do the traces look the same? Should they? (The chip stops at the first mismatch either way.)
5. Type the right password plus one extra character, e.g. `h0px3x`. It **fails**, because the chip also checks that the password ends exactly there.

## 3.4 Lock it in

When the box shows **ACCESS GRANTED**, run the **'Lock it in'** cell. It sends whatever is in the box once more as an independent check and records it as `recovered_pw`.

**YOU SHOULD SEE**

```
hands-on password confirmed: <your password>
```

**IF IT DOESN'T**

`AssertionError: ... did not unlock the target` - the text in the box is wrong. Go back to step 3.2, fix the guess, and run this cell again.

# Part 4 - Change the password

The password lives in the **C source code of the chip's firmware**, not in the notebook. To change it you edit one line, then rebuild and reflash. You might do this to attack a password you chose yourself, or because your instructor wants a different secret for your class.

### Where the password lives

File: `firmware/mcu/basic-passwdcheck/basic-passwdcheck.c` (a sibling of the `jupyter` folder, about line 73):

```
    char correct_passwd[] = "h0px3";
```

At run time the chip compares your guess with this string one byte at a time and quits at the first mismatch - the leak the whole lab exploits. The notebooks only *refer* to the password; they do not store it.

### Rules for a good lab password

| Rule | Why |
|---|---|
| **5 characters long** | The attack loop is written for 5 rounds. (Other lengths work, but need extra edits - see 4.4.) |
| Only `a-z` and `0-9` | The attack only tries those 36 characters. The chip itself would accept others, but the attack would never guess them. |
| Plain keyboard characters, no spaces | Spaces and symbols can be mishandled by the serial link and the attack. |
| Write it down | You will need it for the check in step 5. |

> **WATCH OUT**
> Your copy of the firmware may already contain a password different from `h0px3` (and the original may be commented out with `//` just above). **Look at the line first.** If the notebook's examples show `h0px3` but the attack finds something else, this is why - see step 5.

## 4.1 Path A - the easy way (recommended)

Edit one line, update one check, and re-run the notebook. The setup cell (step 2.2) rebuilds and reflashes the chip for you.

### Step 1 - Let go of the Husky

Make sure nothing else is connected to it (Part 1 warning). Simplest: **Kernel > Shut Down All Kernels** in Jupyter, or close other notebooks.

### Step 2 - Open the firmware file

Open `firmware/mcu/basic-passwdcheck/basic-passwdcheck.c` in a text editor. (It is outside the `jupyter` folder, so use Notepad or VS Code rather than Jupyter.) Press Ctrl+F and search for `correct_passwd[]`.

### Step 3 - Change the text between the quotes

```
BEFORE:    char correct_passwd[] = "h0px3";
AFTER:     char correct_passwd[] = "zzq7a";
```

**WATCH OUT**

Change **only** the characters between the quotation marks. Keep both quotes and the semicolon. If the original is commented out with `//` above the active line, edit the line that does **not** start with `//`. One missing quote or semicolon makes the build fail (T-6).

### Step 4 - Save the file

Ctrl+S. Do not rename it or move it.

### Step 5 - Tell the notebook the new password

Open the MAIN notebook and search it for `h0px3` (Ctrl+F, or Edit > Find in JupyterLab). Update the check cell so it expects your password:

```
BEFORE:  assert guessed_pw == 'h0px3', "Password recovery failed - see ..."
AFTER:   assert guessed_pw == 'zzq7a', "Password recovery failed - see ..."
```

The text blocks that show `h`, `h0`, `h0p`... are just documentation - no need to change them. The Hands-On section needs **no edit**; it follows whatever the attack finds. If your new password is not 5 characters long, also see 4.4.

### Step 6 - Run everything from the top

Use **Kernel > Restart Kernel and Run All Cells** (named 'Restart & Run All' in older versions). The setup cell recompiles the firmware with your new password and flashes it to the chip. Watch the compiler output for the word `error`.

### Step 7 - Prove it worked

**YOU SHOULD SEE**

The attack builds **your** password character by character; the check cell prints `recovered password: zzq7a`; and in the Hands-On box your new password says **ACCESS GRANTED**.

Bonus proof: type the *old* password `h0px3` into the box. It should now say **password fail** - the chip really changed.

## 4.2 Path B - rebuild and reflash by hand (no notebook)

Useful if you only want to reflash the board, or to see what the notebook does behind the scenes. Run these in a terminal opened in the **ChipWhisperer environment** (so `python` has ChipWhisperer):

```
cd chipwhisperer/firmware/mcu/basic-passwdcheck
make PLATFORM=CWHUSKY CRYPTO_TARGET=NONE SS_VER=SS_VER_2_1 -j
python flash_husky.py
```

| Part of the command | What it means |
|---|---|
| `PLATFORM=CWHUSKY` | Build for the Husky + SAM4S kit. **Required** - without it `make` just lists every board it supports and stops. |

| Part of the command | What it means |
|---|---|
| `CRYPTO_TARGET=NONE` | This firmware needs no encryption code. |
| `SS_VER=SS_VER_2_1` | Serial-protocol version; must match the notebook's `SS_VER`. |
| `-j` | Build in parallel (just faster). |
| `python flash_husky.py` | Connects to the Husky and writes the new build to the chip. |

**YOU SHOULD SEE**

The last lines: `Programming firmware...`, then `Firmware flashed successfully!`.

**WATCH OUT**

**Shut down Jupyter kernels first**, or the script cannot connect. Type `python flash_husky.py` exactly, from inside the `basic-passwdcheck` folder - a stray trailing backslash in the path makes PowerShell fail with `can't open file ...flash_husky.py\` (T-9).

The companion notebook `Lab 2_1B - Firmware Change and Flash Walkthrough.ipynb` walks through this same procedure cell by cell and keeps a log of every error hit while building it.

## 4.3 Putting the original password back

Repeat Path A with the original text, and change the notebook's `assert` back:

```
char correct_passwd[] = "h0px3";                              (in basic-passwdcheck.c)
assert guessed_pw == 'h0px3', "Password recovery failed - see ..."   (in the notebook)
```

## 4.4 Using a different length (advanced)

The password does not have to be 5 characters, but then four things must agree:

| Where | Change |
|---|---|
| Firmware | The new password in `correct_passwd[]` (as above). |
| Attack cell | `for _ in range(5):` - replace `5` with the new length. |
| Check cell | The `assert` - use the new password. |
| Capture length | In the `(HARDWARE)` helper notebook: `scope.adc.samples = 3000`. A longer password makes the chip work longer, so later characters may fall beyond the recorded window. Raise the number (try 5000) if the last characters come out wrong. |

**GOOD TO KNOW**

**For instructors:** if you set a new password for a whole class, update it in the firmware, in the `assert` of the MAIN notebook, and in the `assert` of the SOLN notebook, and tell students it was changed - the example text in the notebooks still shows `h0px3`.

# Troubleshooting

Find your symptom, read the likely cause, try the fix. Rows are grouped by where in the lab things go wrong.

### Connection and USB

| # | What you see | Likely cause | Fix |
|---|---|---|---|
| T-1 | `OSError: Unable to communicate with found ChipWhisperer. Check that another process isn't connected...` | Another program (a second notebook, an old script) still holds the Husky. | Kernel > Shut Down All Kernels; close other notebooks; on Windows check for stray `python.exe` in Task Manager; unplug and replug the USB-C. |
| T-2 | `Unable to open USB device`, or no ChipWhisperer found | Missing driver, or a charge-only cable. | Install the WinUSB driver with Zadig (see Lab 0); try another data-capable cable and a different port. |
| T-3 | `SAM3U Serial buffers OVERRUN`, or `LIBUSB_ERROR_NO_DEVICE` part-way through | USB hiccup. | Different cable/port, avoid hubs, then Restart & Run All. |
| T-4 | `AttributeError: 'NoneType' object has no attribute 'clock'` after printing `scope` | The Husky's trace module did not start on this connection. | Unplug the Husky, wait a few seconds, replug, restart the kernel. Avoid printing the raw `scope` object. |

### Building the firmware

| # | What you see | Likely cause | Fix |
|---|---|---|---|
| T-5 | `make` prints a huge list of platforms and ends with `Stop.` | Run by hand without `PLATFORM=`. | Use the full command in 4.2. (The notebook passes it for you.) |
| T-6 | Red `error:` text in the compiler output after you edited the `.c` file | A typo: missing quote, missing `;`, or a stray character. | Re-open the file and compare to the BEFORE/AFTER box in 4.1, step 3. |
| T-7 | `make`, `arm-none-eabi-gcc` or `rm` not found (`CreateProcess ... rm ... failed`) | Compiler tools not on your PATH. | Start Jupyter from the ChipWhisperer compiler environment; re-run Lab 0 check 3. |

### Flashing the chip

| # | What you see | Likely cause | Fix |
|---|---|---|---|
| T-8 | `Serial pins incorrect for NewAE SAM4S target`, then `timeout` | The two serial IO lines were not set (only when flashing by hand). | Use `flash_husky.py` (it sets them), or in the notebook re-run the setup cell. |
| T-9 | Flashing times out even though setup is right; or PowerShell `can't open file '...flash_husky.py\'` | Wrong board mounted (iCE40 FPGA instead of SAM4S) or not seated; for the file error, a trailing backslash. | Check the target says **SAM4S** and is pressed fully onto the CW313. Type `python flash_husky.py` from inside the folder. |

### The attack gives wrong results

| # | What you see | Likely cause | Fix |
|---|---|---|---|
| T-10 | Traces look tiny/flat; the attack prints a nonsense password | The **Measure** SMA cable is loose or disconnected (power signal not reaching the Husky). | Reseat Husky Measure to CW313 MEASURE, finger-tight; also check the ribbon and jumpers; re-run. |
| T-11 | `h`, `hh`, `hhh`, `hhhh`, `hhhhh` | Only if you rewrote the loop: the guess is not using the characters found so far. | Send `guessed_pw + c + "\n"`, not `c + "\n"`. |
| T-12 | `h`, `ha`, `haa`, `haaa` | Only if you rewrote the loop: the reference is not updated each round. | Recompute `ref_trace = cap_pass_trace(guessed_pw + "\x01\n")` inside the loop. |
| T-13 | `AssertionError: Password recovery failed`, but the printed password looks sensible | The chip's password is not `h0px3`. | Update the `assert` (4.1, step 5), or restore the original (4.3). |
| T-14 | The last characters are wrong with a longer password | The recorded window ends before the chip finishes comparing. | Raise `scope.adc.samples` (4.4). |
| T-15 | One middle character differs between runs | Noise on a marginal connection. | Re-run the cell once; if it persists check cable and the Measure SMA. |

### The Hands-On section

| # | What you see | Likely cause | Fix |
|---|---|---|---|
| T-16 | `RuntimeError: try_password() needs the ChipWhisperer connected`, or 'hardware only' messages | `VERSION` is not `'HARDWARE'`, or setup was not run. | Set `VERSION = 'HARDWARE'` and Restart & Run All. |
| T-17 | Box and button do not appear | Stale kernel or browser cache. | Restart kernel, refresh the page, re-run from the top (see 3.2). |
| T-18 | Correct password shows `password fail` | The chip holds a different password than you think; or stale serial data. | Check the line in `basic-passwdcheck.c` and that you reflashed; then Restart & Run All. |
| T-19 | `ModuleNotFoundError: No module named 'ipywidgets'` | Package missing. | `pip install ipywidgets` in the ChipWhisperer environment; restart the kernel. |

# You are done when...

| | Checkpoint |
|---|---|
| [ ] | The setup cell finished and the sanity check printed **OK to continue!** |
| [ ] | You saw the traces for `h` and `0` split apart (step 2.4) and one outlier among the 36 (step 2.5). |
| [ ] | The scoring cell printed one character with a much larger number than the rest (step 2.7). |

| | Checkpoint |
|---|---|
| [ ] | The full attack printed the password growing one character at a time and ended with the whole password (step 2.8). |
| [ ] | The check cell printed `recovered password: ...` with no error. |
| [ ] | In the Hands-On section the recovered password gave **ACCESS GRANTED**, a one-character change gave `password fail`, and the 'Lock it in' cell printed `hands-on password confirmed: ...`. |
| [ ] | *(Optional)* You changed the password, re-ran the lab, and the attack found your new one (Part 4). |

What to hand in, if anything (for example a screenshot or the executed notebook), is up to your instructor - ask before you start.

# Appendix A - Quick reference

| I want to... | Do this |
|---|---|
| **Run a cell** | Click it, press **Shift+Enter**. Wait for `[*]` to turn into a number. |
| **Start fresh** | Kernel > Restart Kernel and Run All Cells (or 'Restart & Run All'). |
| **Free the Husky** | Kernel > Shut Down All Kernels, or close the other notebook tabs. |
| **Find text in the notebook** | Ctrl+F (or Edit > Find). |
| **Start Jupyter** | `cd chipwhisperer/jupyter` then `jupyter lab` |
| **Rebuild the firmware** | `cd chipwhisperer/firmware/mcu/basic-passwdcheck` then `make PLATFORM=CWHUSKY CRYPTO_TARGET=NONE SS_VER=SS_VER_2_1 -j` |
| **Flash it (no notebook)** | `python flash_husky.py` (in that same folder, kernels shut down) |
| **Where is the password?** | `firmware/mcu/basic-passwdcheck/basic-passwdcheck.c`, line starting `char correct_passwd[]` |
| **Stock password** | `h0px3` (5 characters; the attack tries `a-z`, `0-9`) |

# Appendix B - Glossary

| Term | Meaning |
|---|---|
| **Target** | The chip being attacked: the SAM4S microcontroller on the CW312T board. |
| **ChipWhisperer / Husky** | The capture board. It powers the target, talks to it over serial, and records its power use. |
| **Power trace** | A list of numbers: the target's power draw sampled over a short time while it ran. |
| **Trigger** | A signal the firmware raises just before it checks the password, so the Husky starts recording at the right instant. |
| **SPA (simple power analysis)** | Learning secrets by looking at power traces directly, without heavy statistics. |
| **Firmware** | The program that runs on the target chip. Here: `basic-passwdcheck.c`. |
| **Compile / build** | Turn the C source into machine code the chip can run (`make`). |

| Term | Meaning |
|---|---|
| **Flash** | Write that machine code into the chip's memory. |
| **Serial (UART)** | The text connection between the Husky and the chip; how guesses go in and 'PASSWORD FAIL' comes out. |
| **Reset** | Restart the chip. The firmware freezes after each verdict, so the notebook resets it before every guess. |
| **Prefix** | The first few characters of a password, e.g. `h0` is a prefix of `h0px3`. |
| **Notebook, cell, kernel** | The Jupyter page; one runnable block in it; the Python process that runs the cells. |

# Appendix C - Where things live

| File | Folder (from the repository root) and purpose |
|---|---|
| `Lab 2_1B ... (MAIN).ipynb` | In `jupyter/courses/sca101/`. The lab. You work here. |
| `Lab 2_1B ... (HARDWARE).ipynb` | Same folder. Connects, builds, flashes, defines `cap_pass_trace` (run for you). |
| `img/` | Same folder. The reference pictures used in this guide. |
| `basic-passwdcheck.c` | In `firmware/mcu/basic-passwdcheck/`. Chip firmware source. **The only file you edit to change the password.** |
| `basic-passwdcheck-CWHUSKY.hex` | Same folder. The compiled build that gets flashed (regenerated by `make`). |
| `flash_husky.py` | Same folder. Stand-alone flashing script. |


## Notice

This guide accompanies ChipWhisperer course material that references (commercial) training material on ChipWhisperer.io. You can freely execute and use the lab per the open-source license (including in your own courses if you distribute similarly), but you must maintain notice about the source: https://www.ChipWhisperer.io.

Copyright (C) NewAE Technology Inc., 2015-2020. ChipWhisperer is a trademark of NewAE Technology Inc., claimed in all jurisdictions, and registered in at least the United States of America, European Union, and Peoples Republic of China. Tutorials derived from our open-source work must be released under the associated open-source license, and notice of the source must be clearly displayed.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.